

Міністерство освіти і науки України
Національний університет "Львівська політехніка"
Кафедра систем штучного інтелекту



Звіт до лабораторної роботи №1
З дисципліни
"Інтелектуальний аналіз та візуалізація даних"
на тему:
"Вхідний аналіз набору даних"

Виконала:
Студентка групи ІІІ-33
Смачило Іванна
Викладач:
Торська Р.В.

Львів – 2026р.

Тема роботи: Вхідний аналіз набору даних

Мета роботи: Пригадати роботу в Jupyter Notebook, бібліотеці pandas та побудову графіків – і зробити це не на навчальному прикладі, а на реальному файлі, який ніхто не готував спеціально для нас.

Хід роботи

Крок 0: Мої дані та варіант

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ · Смачило Іванна Ростиславівна, ШІ-33, варіант 11

=====

ваш потік: P>10

ваша пара для графіка 3: радіопотік F10.7 та потік P>10

ваш агрегат для етапу 4: середне

ваше додаткове питання: Скільки годин поспіль тривала найдовша неперервна серія

«активних» годин (група з етапу 4)?

Етап 1: Що взагалі лежить у файлі

Завдання 1.1:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df = pd.read_excel("20171129_FINAL_FINAL.xlsx")
df.head()
df.columns
df.dtypes
```

```

Unnamed: 0                                float64
ftp://ftp.swpc.noaa.gov/pub/lists/particle/  object
Unnamed: 2                                object
Unnamed: 3                                object
Unnamed: 4                                object
Unnamed: 5                                object
Unnamed: 6                                object
Unnamed: 7                                object
Unnamed: 8                                object
Unnamed: 9                                object
Unnamed: 10                               object
Unnamed: 11                               object
Unnamed: 12                               object
Unnamed: 13                               object
Unnamed: 14                               object
Unnamed: 15                               float64
Unnamed: 16                               object
Unnamed: 17                               object
Unnamed: 18                               object
Unnamed: 19                               object
Unnamed: 20                               float64
Unnamed: 21                               float64
Unnamed: 22                               float64
Unnamed: 23                               float64
Unnamed: 24                               float64
...
Unnamed: 28                               object
Unnamed: 29                               object
Unnamed: 30                               object
Unnamed: 31                               object
dtype: object

```

Рис. 1.1 Результат зчитування перших рядків і типи стовпців з аркуша

Відповідь 1.1: Звичайне читання дало саме такий результат, бо файл це як одна чиста таблиця. У першому рядку аркуша знаходиться не заголовок стовпців, а текстовий опис або посилання на джерело даних, тому pandas помилково бере цей рядок за назви стовпців. Також в одному аркуші розміщено три різні таблиці, які розділені порожніми стовпцями, а pandas не розпізнає такий розподіл і підписує порожні стовпці як безіменні. Саме через це частина стовпців отримала тип object замість float64/int64. Там змішані текстові підписи й числові дані.

Завдання 1.2:

```
# БАЗ КОД
RAW = pd.read_excel(ФАЙЛ, header=None)
print(RAW.shape)
RAW.head(30)
```

[16] ✓ 0.5s Python

... (7226, 32)

	0	1	2	3	4	5	6	7	8	9	10	11	
0	NaN	ftp://ftp.swpc.noaa.gov/pub/lists/particle/	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
1	NaN		NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
2	NaN	:Data_list: 20170901_Gs_part_5m.txt	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
3	NaN	:Created: 2017 Sep 02 0011 UTC	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
4	NaN	# Prepared by the U.S. Dept. of Commerce, NOAA...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
5	NaN	# Please send comments and suggestions to SWPC...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
6	NaN	#	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
7	NaN	# Label: P > 1 = Particles at >1 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
8	NaN	# Label: P > 5 = Particles at >5 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
9	NaN	# Label: P >10 = Particles at >10 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
10	NaN	# Label: P >30 = Particles at >30 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
11	NaN	# Label: P >50 = Particles at >50 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
12	NaN	# Label: P>100 = Particles at >100 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
13	NaN	# Label: E>0.8 = Electrons at >0.8 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!
14	NaN	# Label: E>2.0 = Electrons at >2.0 Mev	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	!

Рис 1.2 Результат виведення розміру аркуша й перших 30 рядків

Завдання 1.3:

RAW.notna().sum()

[17] ✓ 0.0s

...	0	0
1		7223
2		7202
3		7202
4		7202
5		7203
6		7203
7		7198
8		7198
9		7198
10		7198
11		7198
12		7198
13		7198
14		7198
15		0
16		32
17		26
18		26
19		26
20		0
21		0
22		0
23		0
24		601
...		
28		602
29		602
30		602
31		602

dtype: int64

Рис 1.3 Результат порахованих заповнених клітинок в кожному стовпці

Відповідь 1.3: В аркуші три таблиці, розділені порожніми стовпцями.

Таблиця А займає стовпці 1-14(Excel B:O), таблиця В – стовпці 16-19(Excel Q:T), таблиця С – стовпці 24-31(Excel Y:AF, Y і Z – допоміжний індекс рядка).

Завдання 1.4:

```
# ВАШ КОД
print("Таблица А:")
print(RAW.loc[:24, 1].dropna().to_string(index=False))

[19] ✓ 0.0s

... Таблица А:
ftp://ftp.swpc.noaa.gov/pub/lists/particle/
:Data_list: 20170901_Gs_part_5m.txt
:Created: 2017 Sep 02 0011 UTC
# Prepared by the U.S. Dept. of Commerce, NOAA,...
# Please send comments and suggestions to SWPC...
#
# Label: P > 1 = Particles at >1 Mev
# Label: P > 5 = Particles at >5 Mev
# Label: P >10 = Particles at >10 Mev
# Label: P >30 = Particles at >30 Mev
# Label: P >50 = Particles at >50 Mev
# Label: P>100 = Particles at >100 Mev
# Label: E>0.8 = Electrons at >0.8 Mev
# Label: E>2.0 = Electrons at >2.0 Mev
# Label: E>4.0 = Electrons at >4.0 Mev
# Units: Particles = Protons/cm2-s-sr
# Units: Electrons = Electrons/cm2-s-sr
# Source: GOES-15
# Location: W135
# Missing data: -1.00e+05
5-minute GOES-15 Solar Particle and Electron Flux
# UT
```

Рис 1.4 Службовый опис над таблицей А

```
# ВАШ КОД
print("Таблица В:")
print(RAW.loc[:25, 16].dropna().to_string(index=False))

[20] ✓ 0.0s

... Таблица В:
ftp://ftp.swpc.noaa.gov/pub/indices/old\_indices...
Product: Daily Solar Data quar_DSD.txt
:Issued: 0825 UT 04 Oct 2017
#
# Prepared by the U.S. Dept. of Commerce, NOAA...
# Please send comments and suggestions to SWPC...
Quarterly Daily Solar Data
```

Рис 1.5 Службовый опис над таблицей В

```
# ВАШ КОД
print("Таблица С:")
print(RAW.loc[:25, 26].dropna().to_string(index=False))

[21] ✓ 0.0s

... Таблица С:
http://umtof.umd.edu/pm/crn/
SOHO CELIAS Proton Monitor
Solar Wind Parameters for Carrington Rotation (...
Proton speed [km/sec]
Proton density [protons per cubic centimeter]
Measurement time (Year, Month, Day of Month, Da...
```

Рис 1.6 Службовий опис над таблицею С

Відповідь 1.4:

Таблиця	Джерело	Одиниці вимірювання
А	GOES-15 (ftp://ftp.swpc.noaa.gov/pub/lists/particle/)	Particles = Protons/cm ² -s-sr Electrons = Electrons/cm ² -s-sr
В	Daily Solar Data (ftp://ftp.swpc.noaa.gov/pub/indices/old_indices/2017Q3_DSD.txt)	sfu
С	SOHO CELIAS Proton Monitor (http://umtof.umd.edu/pm/crn/)	Proton speed [km/sec] Proton density [protons per cubic centimeter]

Етап 2: Три таблиці в робочому вигляді

Завдання 2.1:

```

A = pd.read_excel(
    ФАЙЛ,
    sheet_name="Sheet1",
    usecols="B:O",
    skiprows=25,
    header=0,
)
A.columns = ["YR", "MO", "DA", "HHMM", "Julian_Day", "Seconds", "P > 1", "P > 5", "P >10", "P >30", "P >50",
"P>100", "E>0.8", "E>2.0"]
A = A.apply(pd.to_numeric, errors="coerce")
A = A.dropna(how="all")
A = A.reset_index(drop=True)
print(len(A))
A.head()

B = pd.read_excel(
    ФАЙЛ,
    sheet_name="Sheet1",
    usecols="Q:T",
    skiprows=26,
    header=0,
)
B.columns = ["YR", "MO", "DA", "F10_7"]
B = B.apply(pd.to_numeric, errors="coerce")
B = B.dropna(how="all")
B = B.reset_index(drop=True)
print(len(B))
B.head()

C = pd.read_excel(
    ФАЙЛ,
    sheet_name="Sheet1",
    usecols="AA:AF",
    skiprows=26,
    header=0,
)
C.columns = ["YR", "MO", "DA", "HH", "SPEED", "DENSITY"]
C = C.apply(pd.to_numeric, errors="coerce")
C = C.dropna(how="all")
C = C.reset_index(drop=True)
print(len(C))
C.head()

```

7200

25

601

	YR	MO	DA	HH	SPEED	DENSITY
0	17.0	8.0	28.0	0.0	285.0	3.0
1	17.0	8.0	28.0	1.0	305.0	3.4
2	17.0	8.0	28.0	2.0	309.0	1.7
3	17.0	8.0	28.0	3.0	304.0	2.0
4	17.0	8.0	28.0	4.0	302.0	2.3

Рис 2.1 Результат узагальнення стовпців і перетворення значень на числа

Завдання 2.2:

```
hour = A["HHMM"] // 100
minute = A["HHMM"] % 100

A["ts"] = pd.to_datetime(
    dict(year=A["YR"], month=A["MO"], day=A["DA"], hour=hour, minute=minute)
)
print(A["ts"].min(), A["ts"].max())

B["ts"] = pd.to_datetime(dict(year=B["YR"], month=B["MO"], day=B["DA"]))
print(B["ts"].min(), B["ts"].max())

C["ts"] = pd.to_datetime(
    dict(year=C["YR"] + 2000, month=C["MO"], day=C["DA"], hour=C["HH"])
)
print(C["ts"].min(), C["ts"].max())
```

```
2017-08-28 00:00:00 2017-09-21 23:55:00
2017-08-28 00:00:00 2017-09-21 00:00:00
2017-08-28 00:00:00 2017-09-22 00:00:00
```

Рис 2.2 Результати найраніших і найпізніших дат

Самоперевірка етапу 2. Клітинка має виконатися без помилок.

```
sert len(A) == 7200, 'у таблиці A має бути 7200 рядків, а є %d' % len(A)
sert len(B) == 25, 'у таблиці B має бути 25 рядків, а є %d' % len(B)
sert len(C) == 601, 'у таблиці C має бути 601 рядок, а є %d' % len(C)
for назва, t in [('A', A), ('B', B), ('C', C)]:
    рік = pd.to_datetime(t['ts']).dt.year
    assert рік.min() == 2017 and рік.max() == 2017, 'у таблиці %s рік не 2017 – перевірте збирання дати' % назва
print('Етап 2 пройдено')
```

✓ 0.0s

Python

Етап 2 пройдено

Рис 2.3 Самоперевірка етапу 2

Етап 3: Паспорт даних

Завдання 3.1:

Таблиця А:

```
print("Таблиця А:")
print("Розмір:", A.shape)
print("\nТипи стовпців:\n", A.dtypes)
print("\nПропусків:")
print(A.isna().sum().to_string())
print("\nДублікатів рядків:", A.duplicated().sum())
A.describe()
```

```

Таблиця А:
Розмір: (7200, 15)

Типи стовпців:
  YR          int64
  MO          int64
  DA          int64
  HHMM        int64
  Julian_Day  int64
  Seconds     int64
  P > 1       float64
  P > 5       float64
  P >10       float64
  P >30       float64
  P >50       float64
  P>100       float64
  E>0.8       float64
  E>2.0       float64
  ts          datetime64[ns]
dtype: object

Пропусків:
  YR      0
  MO      0
  DA      0
  ...
  E>2.0    3
  ts       0

Дублікатів рядків: 0

```

Рис 3.1.1 Розмір, типи стовпців, кількість пропусків у кожному стовпці, кількість повних дублікатів рядків таблиці А

	YR	MO	DA	HHMM	Julian_Day	Seconds	P > 1	P > 5	P >10
count	7200.0	7200.000000	7200.000000	7200.000000	7200.000000	7200.000000	7197.000000	7197.000000	7197.000000
mean	2017.0	8.840000	13.960000	1177.500000	58005.000000	43050.000000	689.325064	156.626057	76.474020
min	2017.0	8.000000	1.000000	0.000000	57993.000000	0.000000	0.429000	0.151000	0.117000
25%	2017.0	9.000000	7.000000	588.750000	57999.000000	21525.000000	11.700000	0.279000	0.186000
50%	2017.0	9.000000	13.000000	1177.500000	58005.000000	43050.000000	55.400000	0.632000	0.376000
75%	2017.0	9.000000	19.000000	1766.250000	58011.000000	64575.000000	560.000000	107.000000	35.800000
max	2017.0	9.000000	31.000000	2355.000000	58017.000000	86100.000000	32500.000000	2890.000000	1330.000000
std	0.0	0.366632	8.775483	692.481902	7.211603	24943.113498	1625.968447	390.921861	222.362627

Рис 3.1.2 Описові статистики таблиці А

Таблиця В:

```
print("Таблиця В:")
print("Розмір:", B.shape)
print("\nТипи стовпців:\n", B.dtypes)
print("\nПропусків:")
print(B.isna().sum().to_string())
print("\nДублікатів рядків:", B.duplicated().sum())
B.describe()
```

```
Таблиця В:
Розмір: (25, 5)

Типи стовпців:
YR          float64
MO          float64
DA          float64
F10_7       float64
ts          datetime64[ns]
dtype: object

Пропусків:
YR      0
MO      0
DA      0
F10_7   0
ts      0

Дублікатів рядків: 0
```

	YR	MO	DA	F10_7	ts
count	25.0	25.000000	25.000000	25.000000	25
mean	2017.0	8.840000	13.960000	92.680000	2017-09-09 00:00:00
min	2017.0	8.000000	1.000000	71.000000	2017-08-28 00:00:00
25%	2017.0	9.000000	7.000000	74.000000	2017-09-03 00:00:00
50%	2017.0	9.000000	13.000000	84.000000	2017-09-09 00:00:00
75%	2017.0	9.000000	19.000000	107.000000	2017-09-15 00:00:00
max	2017.0	9.000000	31.000000	140.000000	2017-09-21 00:00:00
std	0.0	0.374166	8.955817	22.206831	NaN

Рис 3.1.3 Розмір, типи стовпців, кількість пропусків у кожному стовпці, кількість повних дублікатів рядків таблиці й описові статистики таблиці В

Таблиця С:

```
print("Таблиця С:")
print("Розмір:", C.shape)
print("\nТипи стовпців:\n", C.dtypes)
print("\nПропусків:")
print(C.isna().sum().to_string())
print("\nДублікатів рядків:", C.duplicated().sum())
C.describe()
```

```

Таблиця С:
Розмір: (601, 7)

Типи стовпців:
  YR          float64
  MO          float64
  DA          float64
  HH          float64
  SPEED       float64
  DENSITY     float64
  ts          datetime64[ns]
dtype: object

Пропусків:
  YR      0
  MO      0
  DA      0
  HH      0
  SPEED   0
  DENSITY 0
  ts      0

Дублікатів рядків: 0

```

Рис 3.1.4 Розмір, типи стовпців, кількість пропусків у кожному стовпці, кількість повних дублікатів рядків таблиці С

	YR	MO	DA	HH	SPEED	DENSITY	ts
count	601.0	601.000000	601.000000	601.000000	601.000000	601.000000	601
mean	17.0	8.840266	13.973378	11.480865	506.547421	2.907654	2017-09-09 12:00:00
min	17.0	8.000000	1.000000	0.000000	-1.000000	-1.000000	2017-08-28 00:00:00
25%	17.0	9.000000	7.000000	5.000000	447.000000	1.800000	2017-09-03 06:00:00
50%	17.0	9.000000	13.000000	11.000000	517.000000	2.400000	2017-09-09 12:00:00
75%	17.0	9.000000	19.000000	17.000000	584.000000	3.400000	2017-09-15 18:00:00
max	17.0	9.000000	31.000000	23.000000	806.000000	31.700000	2017-09-22 00:00:00
std	0.0	0.366664	8.781000	6.938063	125.073291	2.637873	NaN

Рис 3.1.5 Описові статистики таблиці С

Завдання 3.2:

```

import numpy as np

speed_mean_before = C["SPEED"].mean()
speed_min_before = C["SPEED"].min()
density_mean_before = C["DENSITY"].mean()
density_min_before = C["DENSITY"].min()

print("SPEED == -1:", (C["SPEED"] == -1).sum())
print("DENSITY == -1:", (C["DENSITY"] == -1).sum())

C.loc[C["SPEED"] < 0, "SPEED"] = np.nan

```

```
C.loc[C["DENSITY"] < 0, "DENSITY"] = np.nan

# "після"
print("\nSPEED: до mean =", speed_mean_before, " min =", speed_min_before)
print("SPEED: після mean =", C["SPEED"].mean(), " min =", C["SPEED"].min())
print("\nDENSITY: до mean =", density_mean_before, " min =", density_min_before)
print("DENSITY: після mean =", C["DENSITY"].mean(), " min =", C["DENSITY"].min())
```

```
SPEED == -1: 8
DENSITY == -1: 8

SPEED: до mean = 506.5474209650582 min = -1.0
SPEED: після mean = 513.3946037099494 min = 277.0

DENSITY: до mean = 2.9076539101497505 min = -1.0
DENSITY: після mean = 2.960370994940978 min = 0.1
```

Рис 3.2 Кількість від'ємних значень

Відповідь 3.2: У таблиці C стовпці SPEED і DENSITY містять по 8 від'ємних значень, а саме -1. Оскільки швидкість і густина – фізичні величини, які не можуть бути від'ємними, ці значення не є реальними вимірюваннями, а є кодом, що дані відсутні. Мінімум після очищення став фізично осмисленим, а середнє трохи зросло, бо штучно занижені значення більше не впливають на розрахунок.

Завдання 3.3:

```
# ВАШ КОД
A[["YR", "MO", "DA", "Julian_Day", "HHMM", "Seconds"]].describe()
```

✓ 0.0s

	YR	MO	DA	Julian_Day	HHMM	Seconds
count	7200.0	7200.000000	7200.000000	7200.000000	7200.000000	7200.000000
mean	2017.0	8.840000	13.960000	58005.000000	1177.500000	43050.000000
std	0.0	0.366632	8.775483	7.211603	692.481902	24943.113498
min	2017.0	8.000000	1.000000	57993.000000	0.000000	0.000000
25%	2017.0	9.000000	7.000000	57999.000000	588.750000	21525.000000
50%	2017.0	9.000000	13.000000	58005.000000	1177.500000	43050.000000
75%	2017.0	9.000000	19.000000	58011.000000	1766.250000	64575.000000
max	2017.0	9.000000	31.000000	58017.000000	2355.000000	86100.000000

Рис 3.3 Результат обрахунку середнього й стандартного відхилення

Відповідь 3.3: Стовпці YR, MO, DA, Julian_Day, HHMM, Seconds – не є результатами вимірювання, а тільки позначення моменту часу, коли було зроблено запис. Рахувати для них середнє арифметичне нема сенсу, бо результати не описують жодного реального явища. На відміну від них, стовпці P>1,...,E>2.0 – реальні фізичні вимірювання і для них середнє арифметичне має змістовний сенс, бо воно показує типовий рівень потоку за період спостереження.

Етап 4: Об'єднання трьох таблиць в одну

Завдання 4.1:

```
# ВАШ КОД
flow_cols = ["P > 1", "P > 5", "P >10", "P >30", "P >50", "P>100", "E>0.8", "E>2.0"]

A_hourly = (
    A.set_index("ts")[flow_cols]
    .resample("h")
    .agg(["mean", "max"])
)
A_hourly.columns = ["_.".join(col).strip() for col in A_hourly.columns]
A_hourly = A_hourly.reset_index()

print(A_hourly.shape)
A_hourly.head()
```

✓ 0.0s Python

(600, 17)

	ts	P > 1_mean	P > 1_max	P > 5_mean	P > 5_max	P >10_mean	P >10_max	P >30_mean	P >30_max	P >50_mean	P >50_max	P >100_mean
0	2017-08-28 00:00:00	9.778333	19.40	0.324917	0.690	0.192917	0.314	0.098850	0.2240	0.077583	0.1780	0.040
1	2017-08-28 01:00:00	7.994167	13.60	0.291667	0.512	0.156583	0.259	0.072583	0.0919	0.058700	0.0798	0.029
2	2017-08-28 02:00:00	6.099167	10.80	0.283000	0.479	0.177667	0.321	0.084450	0.1210	0.069642	0.1110	0.035
3	2017-08-28 03:00:00	4.752500	7.76	0.236833	0.329	0.166333	0.244	0.087658	0.1800	0.068092	0.1390	0.031
4	2017-08-28 04:00:00	4.158333	6.62	0.334917	0.539	0.208333	0.309	0.084800	0.1530	0.072850	0.1420	0.039

Рис 4.1 Зведена таблиця A до погодинної стрілки, середнє і максимум для кожного потоку

Завдання 4.2:

```
import matplotlib.pyplot as plt

день = A_hourly[A_hourly["ts"].dt.date == pd.Timestamp("2017-09-11").date()]

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(день["ts"], день["P >10_mean"], marker="o", label="середнє")
ax.plot(день["ts"], день["P >10_max"], marker="o", label="максимум")

ax.set_xlabel("Час (год)")
ax.set_ylabel("Потік P>10, protons/cm2·s·sr")
ax.set_title("Потік P>10 за 11 вересня 2017: середнє vs максимум за годину")
ax.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

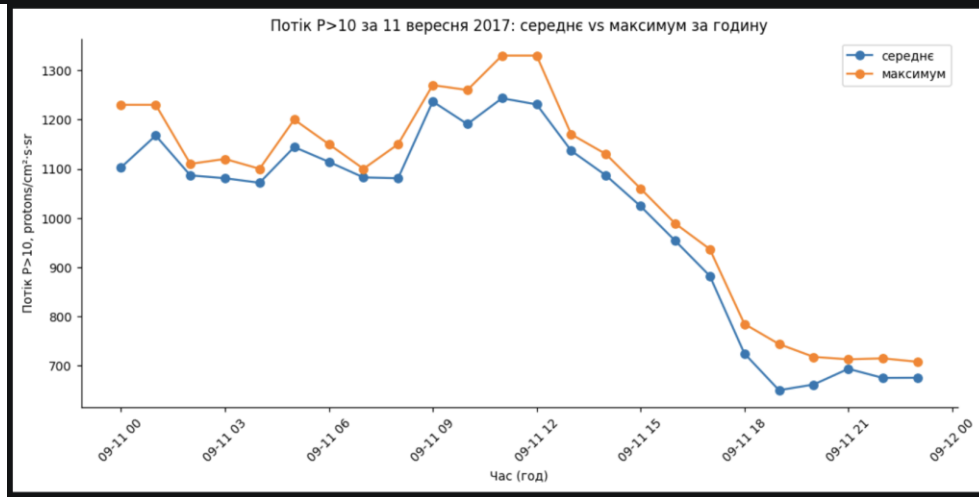


Рис 4.2 Дві криві потоку за різні дні на одному графіку

Відповідь 4.2: Середнє і максимум відрізняються тим, що середнє показує типовий рівень за годину, а максимум показує найвищий одиничний спалах у цій годині. Коли сам потік протягом години поводитьсь відносно рівно, різниця між ними мала. Але коли всередині години стається різкий, короткий стрибок, максимум це вловлює, а середнє – згладжує і різниця між двома кривими зростає. Тому якщо цікавить саме пікове, найнебезпечніше значення, треба дивитись на максимум, а не на середнє.

Завдання 4.3:

```

M_inner = A_hourly.merge(C[["ts", "SPEED", "DENSITY"]], on="ts", how="inner")
M_outer = A_hourly.merge(C[["ts", "SPEED", "DENSITY"]], on="ts", how="outer")

print("inner:", M_inner.shape)
print("outer:", M_outer.shape)

M_inner["date"] = M_inner["ts"].dt.date
B_daily = B[["ts", "F10_7"]].copy()
B_daily["date"] = B_daily["ts"].dt.date

M = M_inner.merge(B_daily[["date", "F10_7"]], on="date", how="left")
M = M.drop(columns="date")

print(M.shape)
M[M.isna().any(axis=1)]

```

inner: (600, 19)
outer: (601, 19)
(600, 20)

	ts	P > 1_mean	P > 1_max	P > 5_mean	P > 5_max	P > 10_mean	P > 10_max	P > 30_mean	P > 30_max	P > 50_mean	P > 50_max	P > 100_mean
175	2017-09-04 07:00:00	6.465833	10.10	0.262083	0.593	0.171500	0.294	0.085483	0.145	0.068558	0.134	0.0
176	2017-09-04 08:00:00	4.010833	5.08	0.256000	0.432	0.195833	0.336	0.086625	0.145	0.070033	0.134	0.0
177	2017-09-04 09:00:00	3.268333	4.42	0.303417	0.518	0.220333	0.302	0.105925	0.162	0.081975	0.134	0.0
178	2017-09-04 10:00:00	3.179167	4.74	0.252167	0.372	0.181833	0.280	0.101900	0.150	0.081675	0.120	0.0
179	2017-09-04 11:00:00	5.274167	7.98	0.295083	0.657	0.186917	0.276	0.089525	0.166	0.063983	0.110	0.0
180	2017-09-04 12:00:00	2.965000	4.85	0.217750	0.428	0.153500	0.239	0.089217	0.179	0.071217	0.144	0.0
181	2017-09-04 13:00:00	3.061667	3.86	0.275833	0.551	0.192833	0.329	0.104542	0.170	0.083783	0.159	0.0
182	2017-09-04 14:00:00	7.279167	10.80	0.231333	0.409	0.170583	0.313	0.086508	0.178	0.070667	0.166	0.0

Рис 4.3 Об'єднана таблиця – M, її розмір і всі рядки, в яких є хоч один пропуск

Відповідь 4.3: Було обрано `how="inner"`, бо `how="outer"` додає лише один зайвий рядок, у якому взагалі немає даних потоку частинок, тобто він не несе жодної корисної інформації для аналізу. Пропуски, які лишаються після об'єднання – справжні пропуски в первинних даних сонячного вітру, а не наслідок невдалого злиття таблиць.

Завдання 4.4:

```
# ВАШ КОД
bins = [-np.inf, 10, 1000, np.inf]
labels = ["тиха", "активна", "дуже активна"]

M["група"] = pd.cut(M["P >10_max"], bins=bins, labels=labels, right=False)

print(M["група"].value_counts())
```

✓ 0.0s

група	
тиха	411
активна	171
дуже активна	18

Name: count, dtype: int64

Рис 4.4 Додана група й скільки годин потрапило в кожену

Самоперевірка етапу 4.

```
assert M is not None, 'таблиця M не побудована'
assert 'ts' in M.columns, 'таблиці M потрібен стовпець ts'
assert 'група' in M.columns, 'таблиці M потрібен стовпець «група»'
assert 590 <= len(M) <= 605, 'очікується близько 600 рядків, є %d – перевірте об'єднання' % len(M)
print('Етап 4 пройдено:', len(M), 'рядків')
```

✓ 0.0s Python

Етап 4 пройдено: 600 рядків

Рис 4.5 Самоперевірка етапу 4

Етап 5: П'ять графіків – п'ять питань

Графік 1: Як розподілені значення вашого потоку?

```
import numpy as np

потік = M["P >10_mean"]

середнє = потік.mean()
медіана = потік.median()
print("середнє:", середнє)
print("медіана:", медіана)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(потік, bins=30, color="#2a78d6")
axes[0].set_xlabel("P>10, protons/cm²·s·sr (середнє за годину)")
```

```

axes[0].set_ylabel("Кількість годин")
axes[0].set_title("Розподіл P>10 (звичайна шкала)")

axes[1].hist(np.log10(потік), bins=30, color="#d6742a")
axes[1].set_xlabel("log10(P>10)")
axes[1].set_ylabel("Кількість годин")
axes[1].set_title("Розподіл log10(P>10)")

plt.tight_layout()
plt.show()

```

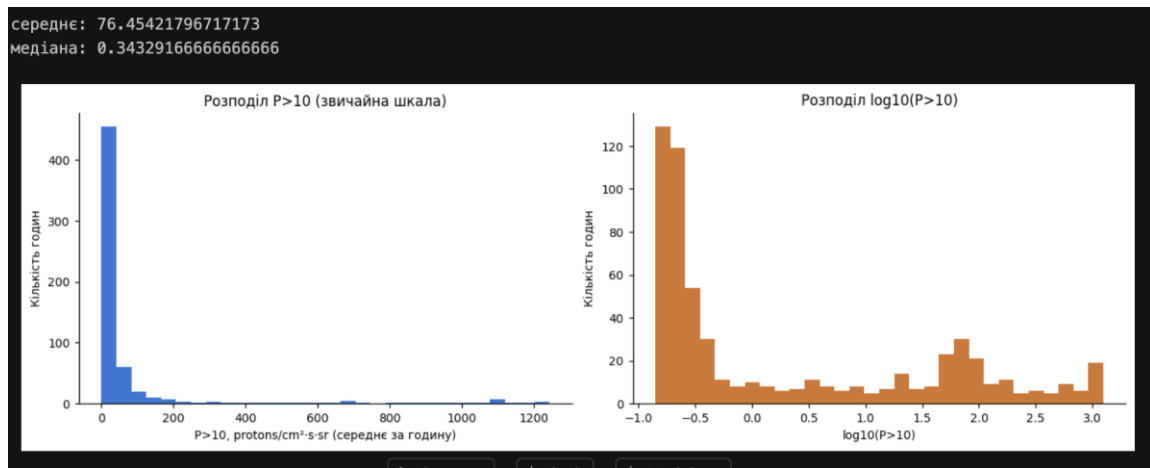


Рис 5.1 Дві гістограми: лінійна й логарифмічна

Висновок: Розподіл потоку $P>10$ дуже нерівномірний: переважна більшість годин мають дуже низьке значення потоку – близько 0.1-0.3, тоді як лише кілька десятків годин демонструють екстремально високі сплески, аж до 1243. Медіана правильно відображає типову годину, бо вона просто вказує на середину впорядкованого ряду й не залежить від того, наскільки великими є викиди. А середнє сильно спотворене цими рідкісними, але величезними значеннями – навіть кілька годин із потоком у сотні й тисячі одиниць буквально тягнуть середнє вгору, роблячи його набагато більшим за медіану. Тому медіана в цьому випадку буде набагато чеснішим показником.

Графік 2: Чи відрізняється швидкість вітру між групами годин?

```
groups = ["тиха", "активна", "дуже активна"]
```

```
data = [M.loc[M["група"] == g, "SPEED"].dropna() for g in groups]

fig, ax = plt.subplots(figsize=(7, 5))
ax.boxplot(data, labels=groups)
ax.set_xlabel("Група годин (за максимумом P>10)")
ax.set_ylabel("Швидкість сонячного вітру, км/с")
ax.set_title("Швидкість сонячного вітру за групами активності")
plt.tight_layout()
plt.show()
```

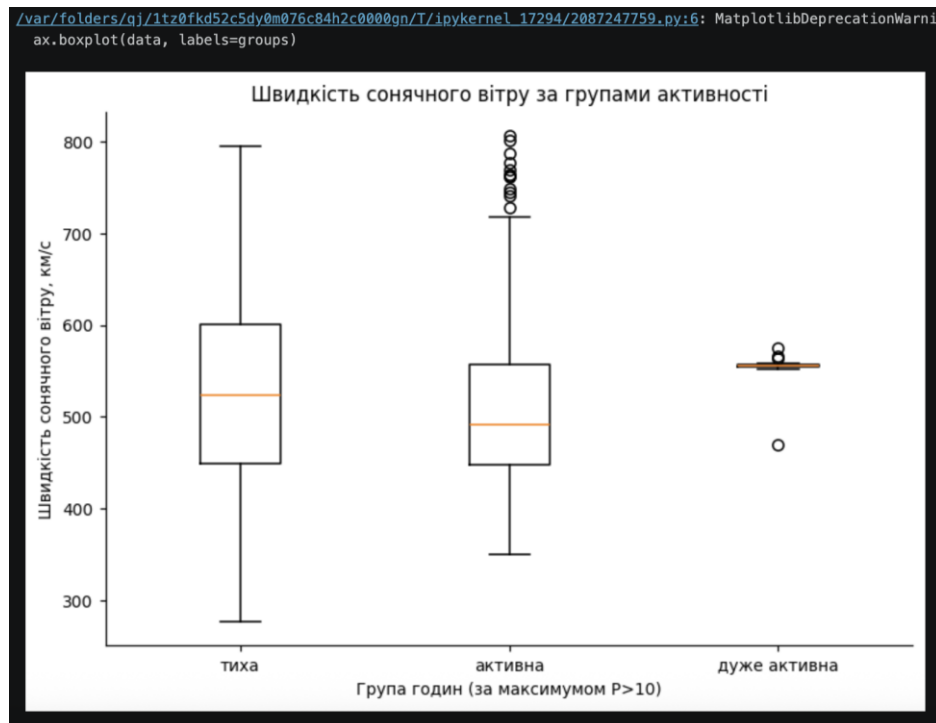


Рис 5.2 Діаграма розмаху швидкості сонячного вітру

Висновок: Медіанна швидкість вітру між групами відрізняється несуттєво. Помітна відмінність – не в рівні, а в розкиді: у дуже активній групі значення значно щільніше згруповані, ніж у тихій чи активній. Отже, за цими даними пряма залежність “швидкість вітру -> активність потоку $P>10$ ” на рівні медіани не простежується.

Графік 3: Чи пов’язані змінні вашої пари?

```

x = M["F10_7"]
y = M["P >10_mean"]

коефіцієнт = x.corr(y)
print("Коефіцієнт кореляції:", коефіцієнт)

fig, ax = plt.subplots(figsize=(7, 5))
ax.scatter(x, y, alpha=0.5, s=15, color="#2a78d6")
ax.set_xlabel("F10.7, sfu")
ax.set_ylabel("P>10, protons/cm²·s·sr (середнє за годину)")
ax.set_title("F10.7 vs P>10 (r = {коефіцієнт:.3f})")
plt.tight_layout()
plt.show()

```

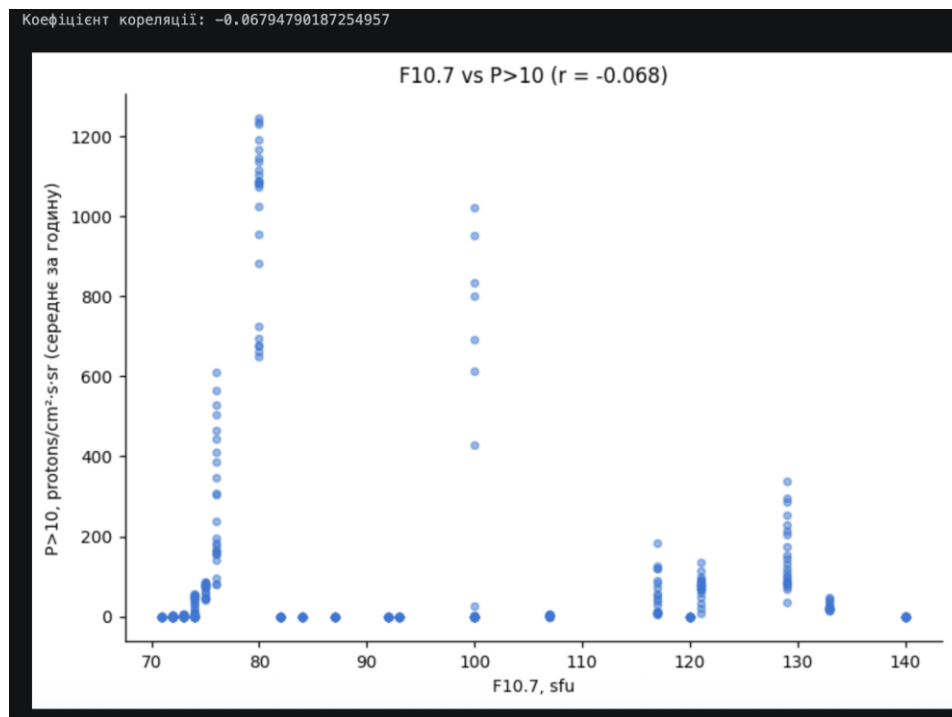


Рис 5.3 Гістограма розсіювання і коефіцієнт кореляції між F10.7 і P>10

Висновок: Кореляція між F10.7 і P>10 дуже слабка і діаграма розсіювання це підтвердила: для одного й того ж значення F10.7 потік P>10 варіюється в діапазоні від майже нуля до понад 1000 одиниць. Це означає, що загальний фоновий рівень сонячної активності не дає змоги передбачити короточасні спалахи протонного потоку – вочевидь, ці спалахи зумовлені окремими подіями, а не поступовою зміною загального рівня активності Сонця.

Графік 4: Скільки годин у кожній групі?

```
підрахунок = M["група"].value_counts().reindex(["тиха", "активна", "дуже активна"])
загалом = підрахунок.sum()

fig, ax = plt.subplots(figsize=(7, 5))
стовпчики = ax.bar(підрахунок.index, підрахунок.values, color="#2a78d6")

for стовпчик, кількість in zip(стовпчики, підрахунок.values):
    відсоток = 100 * кількість / загалом
    ax.text(
        стовпчик.get_x() + стовпчик.get_width() / 2,
        кількість + 5,
        f'{кількість} ({відсоток:.1f}%)',
        ha="center",
    )

ax.set_xlabel("Група годин")
ax.set_ylabel("Кількість годин")
ax.set_title("Розподіл годин за рівнем активності P>10")
plt.tight_layout()
plt.show()
```

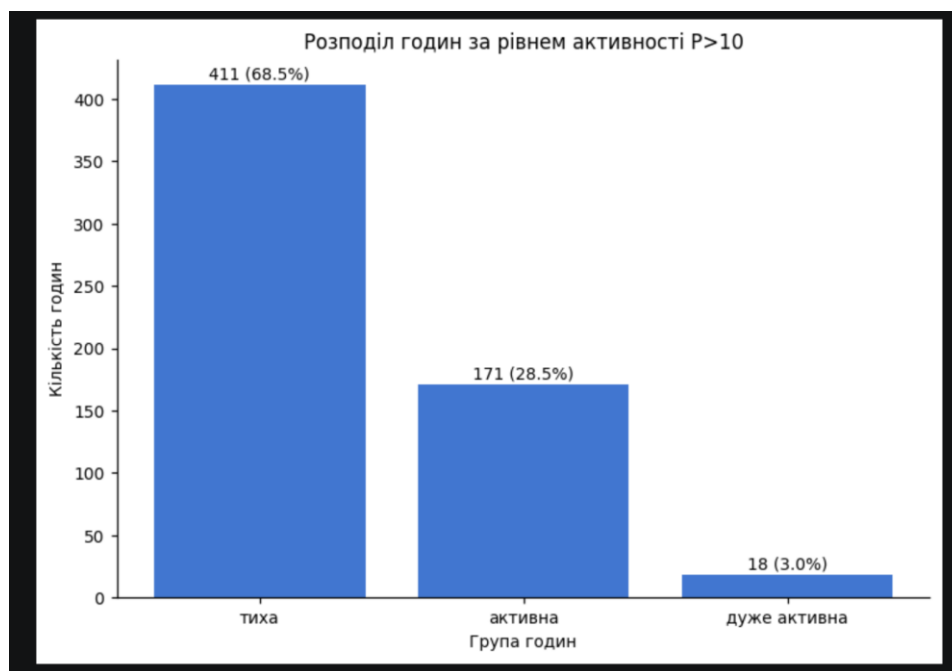


Рис 5.4 Стовпчикова діаграма кількості годин

Висновок: Переважна більшість спостережуваного періоду припадає буквально на тихі години з низьким рівнем протонної активності. Помітно менша частка – активні години, а екстремально високий рівень активності зафіксовано лише у 3% годин, тобто ітенсивні спалахи є рідкісною, короткочасною подією на фоні переважно спокійного періоду.

Графік 5: Як усе це виглядало в часі?

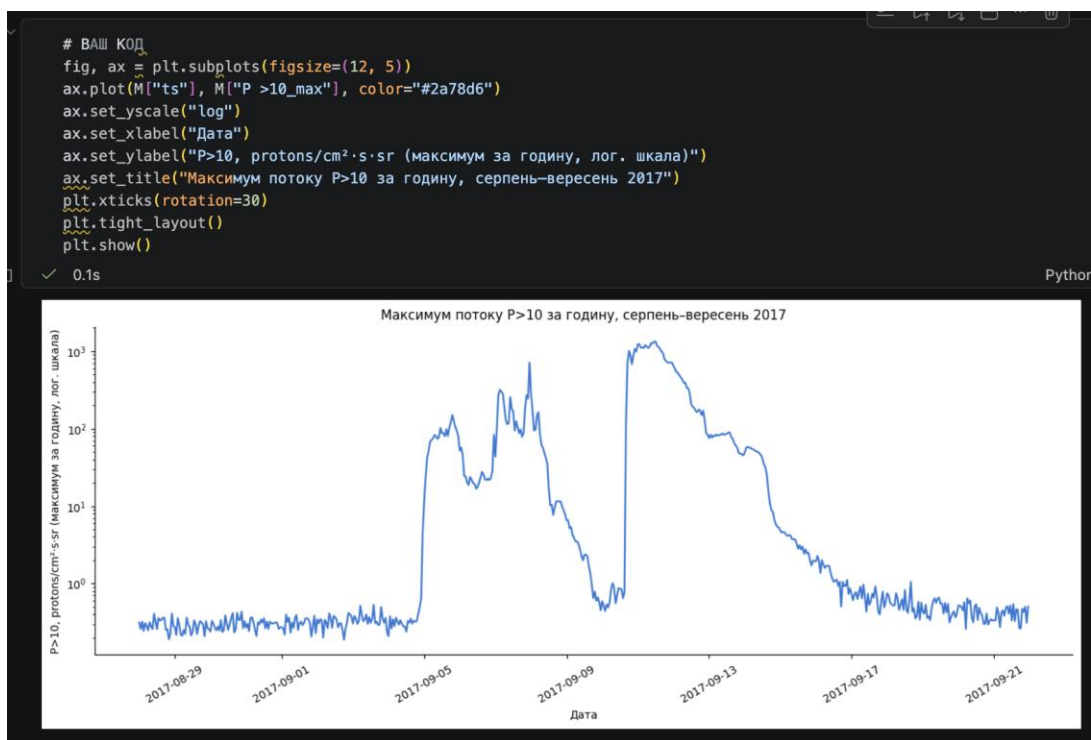


Рис 5.5 Лінійний графік максимуму потоку P>10 за годину за весь період

Висновок: Максимум потоку P>10 протягом усього періоду спостереження не розподілений рівномірно: більшу частину часу він тримається на низькому фоновому рівні, але чітко виділяються два періоди підвищеної активності – приблизно 7-8 вересня і значно потужніший, пікове значення понад 1300 protons/cm²·s·sr, приблизно 10-12 вересня. Це узгоджується з висновком графіка 4: переважна більшість годин тихі, а екстремальна активність зосереджена у вузьких, коротких вікнах часу, а не розмазана рівномірно по всьому періоду.

Етап 6: Два графіки одних і тих самих даних



Рис 6 Задані графіки

Завдання 6.1:

Відповісти на 3 питання по графікам

Відповідь 6.1:

1. Людина, яка бачила тільки Графік 1, зробить висновок, що місяць був спокійним, майже без активності, бо в обраному вузькому вікні справжніх сплесків просто ще не було.
2. Людина, яка бачила тільки Графік 2, побачить весь період і всі сплески, але через логарифмічну шкалу може недооцінити сам масштаб, бо логарифм візуально стискає великі відмінності.
3. Дві відмінності між графіками:
Діапазон часу – вузьке вікно(28.08-05.09) проти всього періоду(28.08-21.09);

Масштаб осі Y – лінійна проти логарифмічної.

Завдання 6.2:

```
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(M["ts"], M["P >10_max"], color="#2a78d6")

пiк = M["P >10_max"].idxmax()
ax.annotate(
    f"пiк: {M.loc[пiк, 'P >10_max']:.0f}",
    xy=(M.loc[пiк, "ts"], M.loc[пiк, "P >10_max"]),
    xytext=(10, -20),
    textcoords="offset points",
)

ax.set_xlabel("Дата")
ax.set_ylabel("P>10, protons/cm2-s-sr (максимум за годину)")
ax.set_title("Майже весь вересень потік P>10 був мінімальним — крім потужного сплеску 10–12 вересня")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



Рис 6.2 Графік, який показує, що відбувається в періоді

Відповідь 6.2: Було обрано весь період спостережень, щоб не приховати сплеск активності, як це сталося на Графіку 1. Обрала також лінійну шкалу, щоб зберегти чесне відчуття масштабу різниці між фоновим рівнем і піком, яке логарифм з Графіку 2 візуально применшує. Натомість пікове значення

підписано прямо на графіку текстом, щоб не втратити цю важливу деталь навіть при лінійній шкалі.

Етап 7: Три спостереження та ваше питання

Завдання 7.1:

Спостереження 1: Розподіл потоку $P > 10$ за годину надзвичайно скошений: середнє(76.45) приблизно у 220 разів більше за медіану(0.34). Це видно на Графіку 1 – на звичайній шкалі майже всі 600 годин втискаються в один стовпчик біля нуля, і лише логарифмічна шкала показує, що насправді розподіл має два горби активності(джерело: Графік 1, Етап 5).

Спостереження 2: Екстремальна активність зосереджена у вузькому вікні часу, а не розмазана по всьому періоду: лише 3% годин(18 з 600) потрапили в групу дуже активної і майже всі вони припадають на 10-12 вересня – саме тоді, коли на Графіку 5 видно найвищий пік потоку(джерело: Графік 4 і Графік 5, Етап 4-5).

Спостереження 3: Загальний фоновий рівень сонячної активності практично не пов'язаний із короткочасними сплесками протонного потоку $P > 10$ – коефіцієнт кореляції становить лише -0.068. Діаграма розсіювання показує, що навіть при однаковому F10.7 потік $P > 10$ може коливатись від майже нуля до понад тисячі(джерело: Графік 3, Етап 5).

Завдання 7.2:

```
# ВАШ КОД
active = M["група"] == "активна"
series = (active != active.shift()).cumsum()
lengths = active.groupby(series).sum()
longest = lengths.max()
print("Найдовша серія активних годин поспіль:", longest)
best = lengths.idxmax()
M[series == best][["ts", "P >10_max", "група"]]
```

✓ 0.0s

Найдовша серія активних годин поспіль: 87

	ts	P >10_max	група
192	2017-09-05 00:00:00	11.4	активна
193	2017-09-05 01:00:00	23.4	активна
194	2017-09-05 02:00:00	41.9	активна
195	2017-09-05 03:00:00	51.3	активна
196	2017-09-05 04:00:00	68.0	активна
...	...	...	...
274	2017-09-08 10:00:00	42.0	активна
275	2017-09-08 11:00:00	35.2	активна
276	2017-09-08 12:00:00	16.6	активна
277	2017-09-08 13:00:00	10.2	активна
278	2017-09-08 14:00:00	10.4	активна

Рис 7.2 Скільки годин поспіль тривала найдовша неперервна серія «активних» годин

Відповідь 7.2: Найдовша неперервна серія годин у групі "активна" тривала 87 годин поспіль(майже 3,6 доби) - з 5 вересня 00:00 до 8 вересня 14:00 2017 року. Це показує, що підвищена активність потоку $P > 10$ могла триматись безперервно кілька діб поспіль, а не лише спалахувати на короткі години.

Етап 8: Декларація про використання інструментів штучного інтелекту

Завдання 8.1:

ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ

=====

автор: Сmacher Іванна Ростиславівна
інструменти: Gemini

етапи 1–2, читання файлу та розбір таблиць: синтаксис
етап 3, паспорт даних і дивні значення: синтаксис
етап 4, об'єднання таблиць: синтаксис
етап 5, побудова графіків: код, перевірено
етап 5, формулювання висновків: код, перевірено
етапи 6–7, порівняння графіків і спостереження: ні

перевірено автором:

ШІ використовувався як консультант для пояснення логіки методів `resample` та `merge`, а також для перевірки синтаксису побудови графіків і форматування висновків.

=====

Автор підтверджує, що може пояснити кожен рядок коду під час захисту.

Крок 9: Збереження результату та фінальна самоперевірка

OK ПІБ і варіант заповнено
OK таблиця A (7200 рядків)
OK таблиця B (25 рядків)
OK таблиця C (601 рядок)
OK таблиці об'єднано
OK стовпець «група» є
OK усі дати у 2017 році
OK декларацію заповнено

Технічна частина роботи виконана.

Тепер переконайтеся, що заповнені ВСІ клітинки з відповідями та висновками.

і виконайте Kernel → Restart & Run All перед здачею.

Висновок: У ході лабораторної роботи №1 я пригадала роботу в Jupyter Notebook, бібліотеці `pandas` та побудову графіків – і зробила це не на навчальному прикладі, а на реальному файлі, який ніхто не готував спеціально для мене.